Supplementary Analysis: Sensitivity of ULB Results to the Raw Time Covariate

Arjun Sekhar

2026-10-04

1. Introduction

1.1. Objective

This notebook tests whether the ULB results reported in the Methodology Notebook depend on the inclusion of raw `Time` as a predictor. In the primary analysis, every model (LR, RF, RSF and Cox) received the same 30 predictors: V1-V28, Amount and `Time`. Here, each model is refitted on identical train-test splits with and without `Time`, so that any difference is attributable to `Time` alone.

1.2. Motivation

Supervisor review (October 2026) asked whether `Time` and the proxy time index used as the survival response are meaningfully different, and whether `Time` explains the strong performance of the tree-based models. Section 3 answers the first question directly. Sections 4 and 5 answer the second by comparing paired fits.

2. Libraries and Data

```
## Step 1: Load required packages (dplyr loaded last to avoid select() masking)
library(survival)
library(randomForestSRC)
library(glmnet)
library(caret)
library(pROC)
library(PRROC)
library(ggplot2)
library(tidyr)
library(pander)
library(dplyr)
```

```
## Step 1: Set seed - same position as Methodology Notebook Section 3.1, so the
## downsampled working dataset below reproduces fraud_subset exactly
set.seed(44940076)

## Step 2: Load data
fraud_data <- read.csv("creditcard.csv", header = TRUE)
fraud_data <- na.omit(fraud_data)

## Step 3: Survival variables (Methodology Notebook Section 5.1)
fraud_data$time   <- 1:nrow(fraud_data)   # proxy time index
fraud_data$status <- fraud_data$Class      # event indicator

## Step 4: Downsampled working dataset (Methodology Notebook Section 5.2)
```

```r
fraud_cases     <- subset(fraud_data, Class == 1)
nonfraud_cases  <- subset(fraud_data, Class == 0)
nonfraud_sample <- nonfraud_cases[sample(nrow(nonfraud_cases), 50000), ]
fraud_subset    <- rbind(fraud_cases, nonfraud_sample)

dim(fraud_subset)
```

```
## [1] 50492    33
```

```r
table(fraud_subset$Class)
```

```
##
##     0     1
## 50000   492
```

3. Time-Covariate Audit

```r
## Step 1: Does raw Time carry the same ordering as the proxy index?
audit_time_sorted     <- !is.unsorted(fraud_data$Time)
audit_spearman_full   <- cor(fraud_data$Time, fraud_data$time, method = "spearman")
audit_spearman_subset <- cor(fraud_subset$Time, fraud_subset$time, method = "spearman")

## Step 2: One stratified 70-30 split for the model-based checks
set.seed(44940076)
audit_idx   <- createDataPartition(fraud_subset$status, p = 0.7, list = FALSE)
audit_train <- fraud_subset[audit_idx, ]
audit_test  <- fraud_subset[-audit_idx, ]

## Step 3: Time on its own - survival outcome vs fraud outcome.
## concordance() treats larger predictor = longer survival by default.
audit_c_time   <- concordance(Surv(time, status) ~ Time, data = audit_test)$concordance
audit_auc_time <- as.numeric(auc(roc(response = audit_test$status,
                                     predictor = audit_test$Time, quiet = TRUE)))

## Step 4: Unpenalised Cox with vs without Time, capturing warnings
fit_cox_capture <- function(predictors, data) {
  warn_msgs <- character(0)
  cox_formula <- as.formula(paste("Surv(time, status) ~", paste(predictors, collapse = " + ")))
  fit <- withCallingHandlers(
    coxph(cox_formula, data = data),
    warning = function(w) {
      warn_msgs <<- c(warn_msgs, conditionMessage(w))
      invokeRestart("muffleWarning")
    }
  )
  list(fit = fit, warnings = warn_msgs)
}

audit_cox_with    <- fit_cox_capture(c(paste0("V", 1:28), "Amount", "Time"), audit_train)
audit_cox_without <- fit_cox_capture(c(paste0("V", 1:28), "Amount"), audit_train)

## Step 5: Plain named values for inline reporting
audit_n_warn_with    <- length(audit_cox_with$warnings)
audit_n_warn_without <- length(audit_cox_without$warnings)
```

```
audit_iter_without   <- audit_cox_without$fit$iter
audit_time_logHR_sd  <- unname(coef(audit_cox_with$fit)["Time"]) * sd(audit_train$Time)

## Step 6: Summary tables
pander(data.frame(
  Check = c("Full data already sorted by Time",
            "Spearman(Time, proxy index) - full data",
            "Spearman(Time, proxy index) - working subset",
            "Harrell's C, Time alone (survival outcome)",
            "AUC, Time alone (fraud vs non-fraud)"),
  Value = c(as.character(audit_time_sorted), round(audit_spearman_full, 6),
            round(audit_spearman_subset, 6), round(audit_c_time, 4), round(audit_auc_time, 4))
), caption = "Raw Time vs Proxy Time Index")
```

Table 1: Raw Time vs Proxy Time Index

Check	Value
Full data already sorted by Time	TRUE
Spearman(Time, proxy index) - full data	1
Spearman(Time, proxy index) - working subset	1
Harrell's C, Time alone (survival outcome)	1
AUC, Time alone (fraud vs non-fraud)	0.5979

```
pander(data.frame(
  Specification     = c("With Time", "Without Time"),
  n_warnings        = c(audit_n_warn_with, audit_n_warn_without),
  iterations        = c(audit_cox_with$fit$iter, audit_iter_without),
  Time_logHR_per_SD = c(round(audit_time_logHR_sd, 3), NA)
), caption = "Unpenalised Cox: With vs Without Raw Time (Single Split)")
```

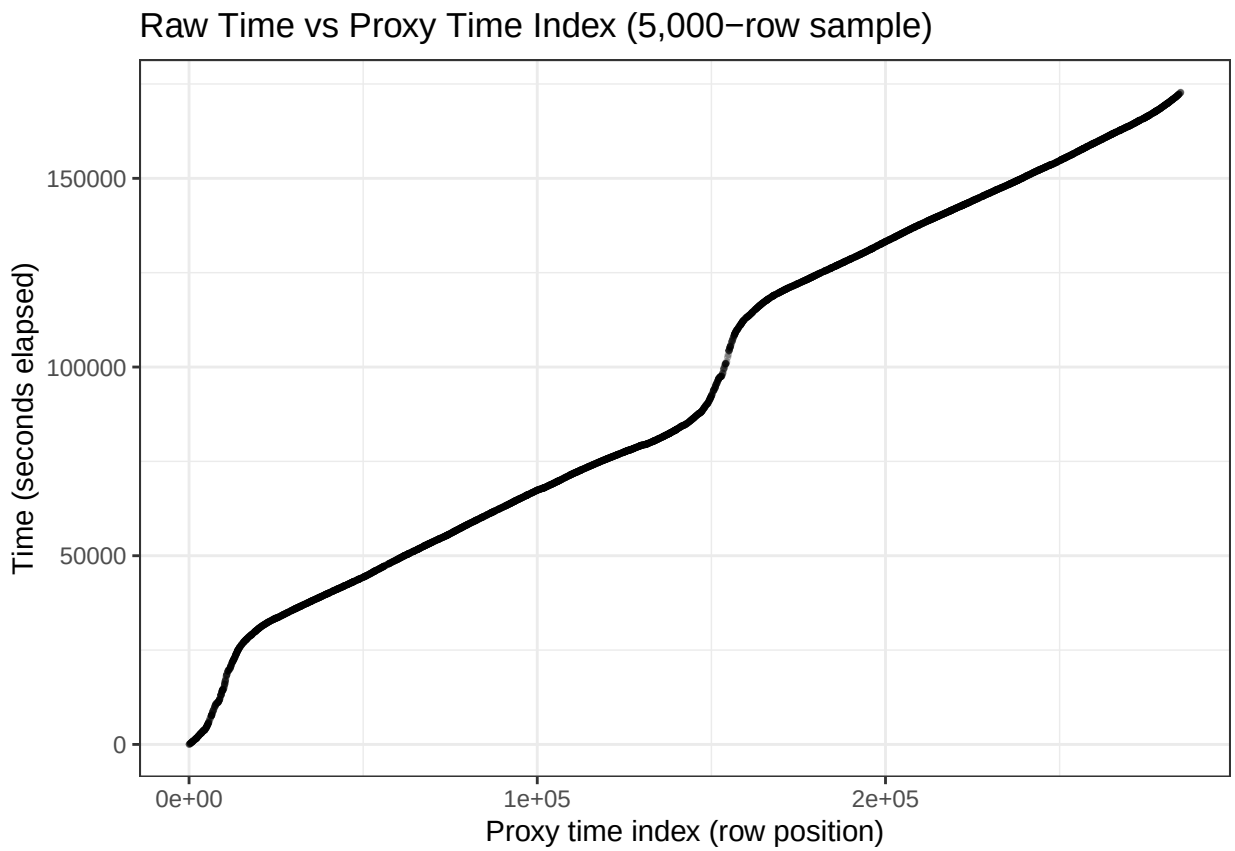
Table 2: Unpenalised Cox: With vs Without Raw Time (Single Split)

Specification	n_warnings	iterations	Time_logHR_per_SD
With Time	2	21	-332.4
Without Time	0	13	NA

```
audit_cox_with$warnings
```

```
## [1] "Ran out of iterations and did not converge"
## [2] "one or more coefficients may be infinite"
```

```
## Step 1: Visual check on a 5,000-row sample
set.seed(44940076)
plot_idx <- sample(nrow(fraud_data), 5000)
ggplot(fraud_data[plot_idx, ], aes(x = time, y = Time)) +
  geom_point(alpha = 0.3, size = 0.6) +
  labs(title = "Raw Time vs Proxy Time Index (5,000-row sample)",
       x = "Proxy time index (row position)", y = "Time (seconds elapsed)") +
  theme_bw()
```



Observations: Raw `Time` and the proxy index have a Spearman correlation of 1 on the full dataset and 1 on the working subset, because the ULB file is already sorted by `Time`; the only difference is that the clock advances faster per row overnight, when transaction volume is low. On its own, `Time` gives a Harrell's C of 1 against the survival outcome but an AUC of only 0.598 for separating fraud from non-fraud. An unpenalised Cox model with `Time` returned 2 warnings with a log-hazard ratio of -332.4 per SD of `Time`, while the model without `Time` converged in 13 iterations with no warnings. The non-convergence reported in the Methodology Notebook (Section 11.1) is therefore predominantly driven by `Time`, although Section 4.6 shows that one of 30 splits still fails to converge without it.

4. Paired Sensitivity Analysis

4.1. Design

Each model is fitted twice on every split: once with the primary predictor set (V1-V28, Amount, `Time`) and once without `Time`. The same 30 stratified 70-30 splits are reused across all models and both specifications, so the comparison is fully paired. Model settings match the Methodology Notebook (no SMOTE, downsampled working dataset). Thresholds follow the same rule: the 0.99 quantile of training-set scores.

```
## Step 1: Predictor sets
pred_with    <- c(paste0("V", 1:28), "Amount", "Time")
pred_without <- c(paste0("V", 1:28), "Amount")
pred_sets    <- list(with_Time = pred_with, without_Time = pred_without)

## Step 2: Fix 30 stratified splits up front, shared by every model
n_splits <- 30
set.seed(44940076)
split_idx <- lapply(1:n_splits, function(i)
```

```
  as.vector(createDataPartition(fraud_subset$status, p = 0.7, list = FALSE)))

## Step 3: Metric helper - AUC, AUPRC, and sensitivity/precision at the
## 0.99 training-quantile threshold (Methodology Notebook convention)
eval_scores <- function(train_score, test_score, test_labels, threshold_q = 0.99) {
  auc_val <- as.numeric(auc(roc(response = test_labels, predictor = test_score, quiet = TRUE)))
  auprc_val <- pr.curve(scores.class0 = test_score[test_labels == 1],
                        scores.class1 = test_score[test_labels == 0],
                        curve = FALSE)$auc.integral
  thr  <- quantile(train_score, probs = threshold_q, na.rm = TRUE)
  pred <- as.integer(test_score > thr)
  tp <- sum(pred == 1 & test_labels == 1)
  fp <- sum(pred == 1 & test_labels == 0)
  fn <- sum(pred == 0 & test_labels == 1)
  data.frame(AUC = auc_val, AUPRC = auprc_val,
             Sensitivity_099 = tp / (tp + fn),
             Precision_099   = if ((tp + fp) == 0) NA_real_ else tp / (tp + fp))
}
```

4.2. Logistic Regression

```
## Step 1: Fit LR with and without Time on each split
set.seed(44940076)
lr_rows <- list()
for (i in 1:n_splits) {
  train_i <- fraud_subset[split_idx[[i]], ]
  test_i  <- fraud_subset[-split_idx[[i]], ]
  for (spec in names(pred_sets)) {
    lr_fit <- glm(reformulate(pred_sets[[spec]], response = "status"),
                  data = train_i, family = binomial)
    lr_train_score <- predict(lr_fit, newdata = train_i, type = "response")
    lr_test_score  <- predict(lr_fit, newdata = test_i,  type = "response")
    lr_rows[[length(lr_rows) + 1]] <- cbind(
      data.frame(split = i, model = "LR", spec = spec, concordance = NA_real_),
      eval_scores(lr_train_score, lr_test_score, test_i$status))
  }
}

## Step 2: Save
saveRDS(do.call(rbind, lr_rows), "time_sens_lr.rds")
```

4.3. Random Forest (Classification)

```
## Step 1: Fit RF with and without Time on each split (settings as Section 10.1)
set.seed(44940076)
rf_rows <- list()
for (i in 1:n_splits) {
  message("RF split ", i, " of ", n_splits, " - ", Sys.time())
  train_i <- fraud_subset[split_idx[[i]], ]
  test_i  <- fraud_subset[-split_idx[[i]], ]
  train_i$Class <- factor(train_i$Class, levels = c(0, 1))
  for (spec in names(pred_sets)) {
    rf_fit <- rfsrc(reformulate(pred_sets[[spec]], response = "Class"),
```

```
                    data = train_i, ntree = 100, nodesize = 15,
                    splitrule = "gini", nthread = 4, fast = TRUE)
    rf_train_score <- predict(rf_fit,
                              newdata = train_i[, rf_fit$xvar.names, drop = FALSE])$predicted[, "1"]
    rf_test_score  <- predict(rf_fit,
                              newdata = test_i[, rf_fit$xvar.names, drop = FALSE])$predicted[, "1"]
    rf_rows[[length(rf_rows) + 1]] <- cbind(
      data.frame(split = i, model = "RF", spec = spec, concordance = NA_real_),
      eval_scores(rf_train_score, rf_test_score, test_i$status))
  }
}

## Step 2: Save
saveRDS(do.call(rbind, rf_rows), "time_sens_rf.rds")
```

4.4. Random Survival Forest

```
## Step 1: Fit RSF with and without Time on each split (settings as Section 7.4)
set.seed(44940076)
rsf_rows <- list()
for (i in 1:n_splits) {
  message("RSF split ", i, " of ", n_splits, " - ", Sys.time())
  train_i <- fraud_subset[split_idx[[i]], ]
  test_i  <- fraud_subset[-split_idx[[i]], ]
  for (spec in names(pred_sets)) {
    rsf_fit <- rfsrc(as.formula(paste("Surv(time, status) ~",
                                      paste(pred_sets[[spec]], collapse = " + "))),
                     data = train_i, ntree = 100, nodesize = 15,
                     splitrule = "logrank", nthread = 4, fast = TRUE)
    ## Risk score: 1 - S(tmax), as in the Methodology Notebook
    rsf_train_pred  <- predict(rsf_fit, newdata = train_i[, rsf_fit$xvar.names, drop = FALSE])
    rsf_test_pred   <- predict(rsf_fit, newdata = test_i[,  rsf_fit$xvar.names, drop = FALSE])
    rsf_train_score <- as.numeric(1 - rsf_train_pred$survival[, ncol(rsf_train_pred$survival)])
    rsf_test_score  <- as.numeric(1 - rsf_test_pred$survival[,  ncol(rsf_test_pred$survival)])
    rsf_conc <- concordance(Surv(time, status) ~ rsf_test_score,
                            data = test_i, reverse = TRUE)$concordance
    rsf_rows[[length(rsf_rows) + 1]] <- cbind(
      data.frame(split = i, model = "RSF", spec = spec, concordance = rsf_conc),
      eval_scores(rsf_train_score, rsf_test_score, test_i$status))
  }
}

## Step 2: Save
saveRDS(do.call(rbind, rsf_rows), "time_sens_rsf.rds")
```

4.5. Cox Proportional Hazards (Ridge)

```
## Step 1: Fit ridge Cox with and without Time on each split (as Section 11.1)
set.seed(44940076)
cox_rows <- list()
for (i in 1:n_splits) {
  message("Cox split ", i, " of ", n_splits, " - ", Sys.time())
  train_i <- fraud_subset[split_idx[[i]], ]
```

```
  test_i  <- fraud_subset[-split_idx[[i]], ]
  y_train <- Surv(train_i$time, train_i$status)
  for (spec in names(pred_sets)) {
    x_train <- as.matrix(train_i[, pred_sets[[spec]]])
    x_test  <- as.matrix(test_i[,  pred_sets[[spec]]])
    cox_cv  <- cv.glmnet(x_train, y_train, family = "cox", alpha = 0)
    cox_fit <- glmnet(x_train, y_train, family = "cox", alpha = 0, lambda = cox_cv$lambda.min)
    cox_train_score <- as.numeric(predict(cox_fit, newx = x_train, type = "link"))
    cox_test_score  <- as.numeric(predict(cox_fit, newx = x_test,  type = "link"))
    cox_conc <- concordance(Surv(time, status) ~ cox_test_score,
                            data = test_i, reverse = TRUE)$concordance
    cox_rows[[length(cox_rows) + 1]] <- cbind(
      data.frame(split = i, model = "Cox (ridge)", spec = spec, concordance = cox_conc),
      eval_scores(cox_train_score, cox_test_score, test_i$status))
  }
}

## Step 2: Save
saveRDS(do.call(rbind, cox_rows), "time_sens_cox_ridge.rds")
```

4.6. Cox Proportional Hazards (Unpenalised, Without Time)

The unpenalised model is fitted without `Time` only, since Section 3 shows it does not converge with `Time` included. Warnings are counted per split rather than halting the loop.

```
## Step 1: Fit unpenalised Cox without Time on each split
set.seed(44940076)
cox_u_rows <- list()
cox_u_formula <- as.formula(paste("Surv(time, status) ~", paste(pred_without, collapse = " + ")))
for (i in 1:n_splits) {
  train_i <- fraud_subset[split_idx[[i]], ]
  test_i  <- fraud_subset[-split_idx[[i]], ]
  n_warn  <- 0L
  cox_u_fit <- withCallingHandlers(
    coxph(cox_u_formula, data = train_i),
    warning = function(w) { n_warn <<- n_warn + 1L; invokeRestart("muffleWarning") }
  )
  cox_u_train_score <- as.numeric(predict(cox_u_fit, newdata = train_i, type = "lp"))
  cox_u_test_score  <- as.numeric(predict(cox_u_fit, newdata = test_i,  type = "lp"))
  cox_u_rows[[length(cox_u_rows) + 1]] <- cbind(
    data.frame(split = i, model = "Cox (unpenalised)", spec = "without_Time",
               concordance = concordance(cox_u_fit, newdata = test_i)$concordance,
               n_warnings = n_warn),
    eval_scores(cox_u_train_score, cox_u_test_score, test_i$status))
}

## Step 2: Save
saveRDS(do.call(rbind, cox_u_rows), "time_sens_cox_unpen.rds")
```

5. Results

5.1. Summary by Model and Specification

```
## Step 1: Load and combine all split-level results
cox_unpen_results <- readRDS("time_sens_cox_unpen.rds")
sens_results <- dplyr::bind_rows(
  readRDS("time_sens_lr.rds"),
  readRDS("time_sens_rf.rds"),
  readRDS("time_sens_rsf.rds"),
  readRDS("time_sens_cox_ridge.rds"),
  cox_unpen_results %>% dplyr::select(-n_warnings)
)

## Step 2: Mean and SD across the 30 splits
sens_summary <- sens_results %>%
  dplyr::group_by(model, spec) %>%
  dplyr::summarise(
    AUC_mean         = mean(AUC),             AUC_SD   = sd(AUC),
    AUPRC_mean       = mean(AUPRC),           AUPRC_SD = sd(AUPRC),
    Sensitivity_099  = mean(Sensitivity_099),
    Precision_099    = mean(Precision_099, na.rm = TRUE),
    Concordance_mean = mean(concordance),
    .groups = "drop"
  )

pander(sens_summary, digits = 4, split.table = Inf,
       caption = "Performance With and Without Raw Time (30 Paired Splits, No SMOTE)")
```

Table 3: Performance With and Without Raw Time (30 Paired Splits, No SMOTE)

model	spec	AUC_mean	AUC_SD	AUPRC_mean	AUPRC_SD	Sensitivity_099	Precision_099	Concordance_mean
Cox (ridge)	with_Time	0.9392	0.009706	0.6701	0.03762	0.6662	0.6471	0.9902
Cox (ridge)	without_Time	0.9664	0.00848	0.7791	0.03694	0.7628	0.7491	0.9747
Cox (unpenalised)	without_Time	0.9451	0.009772	0.6791	0.04072	0.6735	0.6518	0.9665
LR	with_Time	0.9795	0.005475	0.8661	0.02069	0.8595	0.8247	NA
LR	without_Time	0.9791	0.006006	0.8656	0.02086	0.8597	0.8234	NA
RF	with_Time	0.9753	0.006737	0.8811	0.01922	0.8579	0.863	NA
RF	without_Time	0.9759	0.006566	0.881	0.01941	0.8582	0.8641	NA
RSF	with_Time	0.9742	0.00642	0.858	0.02358	0.8481	0.8806	0.9775
RSF	without_Time	0.9745	0.007659	0.858	0.0208	0.8533	0.8673	0.9746

```
## Step 3: Convergence of the unpenalised Cox model
cox_unpen_converged <- sum(cox_unpen_results$n_warnings == 0)
cox_unpen_converged
```

```
## [1] 29
```

5.2. Paired Differences

```r
## Step 1: Paired AUPRC and AUC differences (without Time minus with Time)
paired_diff <- sens_results %>%
  dplyr::filter(model %in% c("LR", "RF", "RSF", "Cox (ridge)")) %>%
  dplyr::select(split, model, spec, AUC, AUPRC, concordance) %>%
  tidyr::pivot_wider(names_from = spec, values_from = c(AUC, AUPRC, concordance))

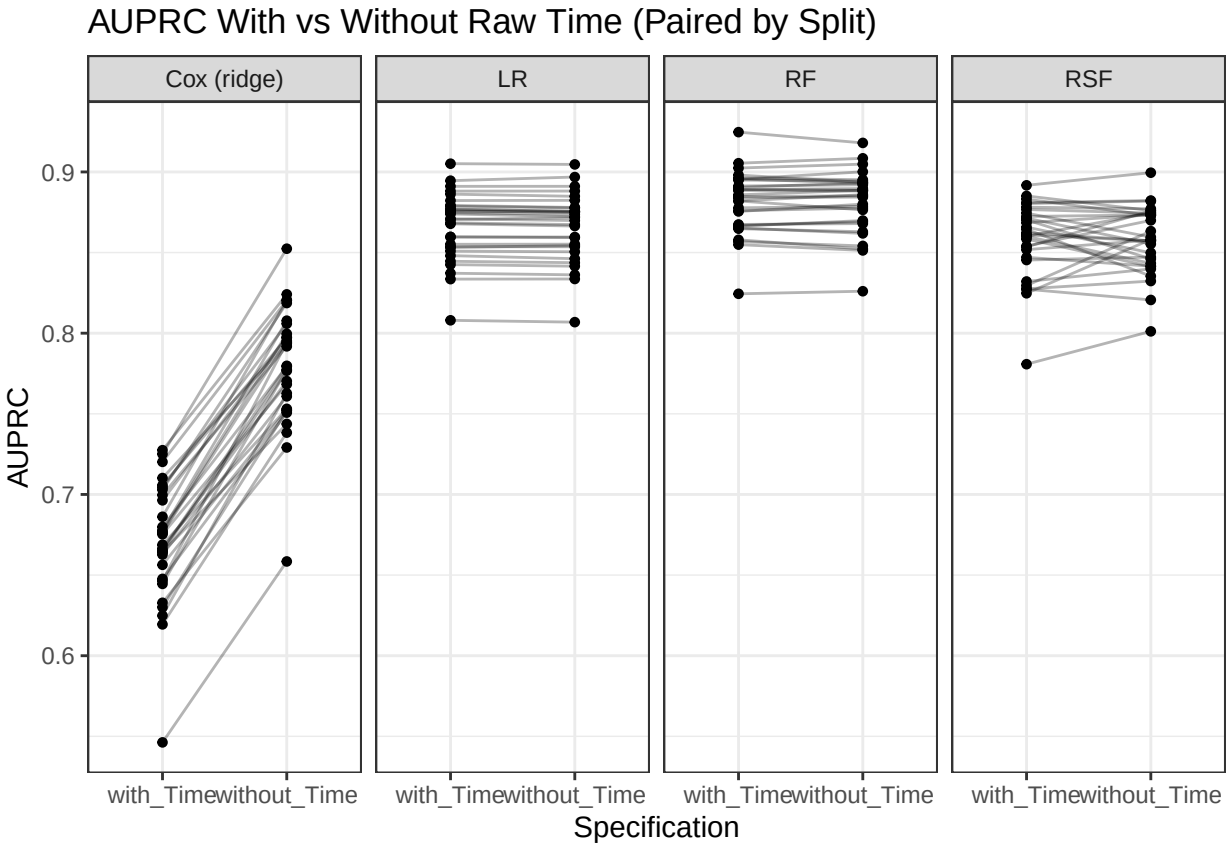
paired_tests <- paired_diff %>%
  dplyr::group_by(model) %>%
  dplyr::summarise(
    AUPRC_with      = mean(AUPRC_with_Time),
    AUPRC_without   = mean(AUPRC_without_Time),
    AUPRC_diff      = mean(AUPRC_without_Time - AUPRC_with_Time),
    AUPRC_wilcox_p  = wilcox.test(AUPRC_without_Time, AUPRC_with_Time,
                                  paired = TRUE, exact = FALSE)$p.value,
    AUC_diff        = mean(AUC_without_Time - AUC_with_Time),
    Concordance_diff = mean(concordance_without_Time - concordance_with_Time),
    .groups = "drop"
  )

pander(paired_tests, digits = 4, split.table = Inf,
       caption = "Paired Effect of Removing Time (Without minus With, 30 Splits)")
```

Table 4: Paired Effect of Removing Time (Without minus With, 30 Splits)

model	AUPRC_with	AUPRC_without	AUPRC_diff	AUPRC_wilcox_p	AUC_diff	Concordance_diff
Cox (ridge)	0.6701	0.7791	0.109	1.825e-06	0.02712	-0.01544
LR	0.8661	0.8656	-0.0005391	0.004534	-0.0003388	NA
RF	0.8811	0.881	-8.679e-05	0.8855	0.0006067	NA
RSF	0.858	0.858	5.547e-08	0.805	0.0003142	-0.002919

```r
## Step 1: Paired AUPRC by split, with vs without Time
ggplot(sens_results %>% dplyr::filter(model != "Cox (unpenalised)"),
       aes(x = spec, y = AUPRC, group = split)) +
  geom_line(alpha = 0.3) +
  geom_point(size = 1.2) +
  facet_wrap(~ model, nrow = 1) +
  labs(title = "AUPRC With vs Without Raw Time (Paired by Split)",
       x = "Specification", y = "AUPRC") +
  theme_bw()
```



Observations: Removing `Time` leaves the tree-based models effectively unchanged: RF AUPRC moves from 0.8811 to 0.8810 (Wilcoxon $p = 0.886$) and RSF from 0.8580 to 0.8580 ($p = 0.805$). LR shows a statistically significant but negligible decrease of 0.0005 ($p = 0.005$), which reflects consistent tiny differences across paired splits rather than a meaningful effect. The RSF-vs-LR and RF-vs-LR gaps are therefore unchanged, and `Time` does not explain the tree models' performance.

Ridge Cox is the exception: AUPRC rises from 0.670 to 0.779 ($p < 0.001$) and AUC from 0.939 to 0.966 once `Time` is removed, indicating that `Time` dominated the linear risk score while carrying little fraud signal (AUC 0.598 on its own). Concordance falls only slightly for both survival models (Cox -0.015, RSF -0.003) and remains near 0.975, so the high concordance reported in the Methodology Notebook (Section 11.2) mainly reflects the event-censoring structure rather than `Time`. Unpenalised Cox without `Time` converged on 29 of 30 splits and performed well below ridge (AUPRC 0.679 vs 0.779), so the ridge specification remains appropriate.

6. Synthesis

The primary ULB conclusions are robust to removing raw `Time`. LR, RF and RSF performance is unchanged, so the null result for RSF versus LR does not depend on `Time`, and `Time` does not account for the tree-based models' performance. The only material effect is on Cox, whose AUPRC improves from 0.670 to 0.779 without `Time`. Cox's weaker performance in the primary analysis is therefore partly attributable to `Time` dominating its risk score, although it remains the lowest-performing model under either specification.